

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 4 – Laboratory Report
Comparative Study of Deep Convolutional Neural
Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: T. A. Prasanna

Roll No.: 24110124

Registration No.: 24011101113

Github: <https://github.com/Prasannnnnnnnnaaaaa/Deep-Learning-Lab.git>

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment, the following outcomes were achieved.

1. Explained modern CNN architectures and their design motivations.
2. Differentiated LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implemented transfer learning using a pretrained ImageNet backbone.
4. Fine tuned a pretrained CNN model by selectively unfreezing convolution layers.
5. Compared CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance. This report implements and compares these architectures on the CIFAR-10 dataset using transfer learning.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogLeNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$\text{Output} = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

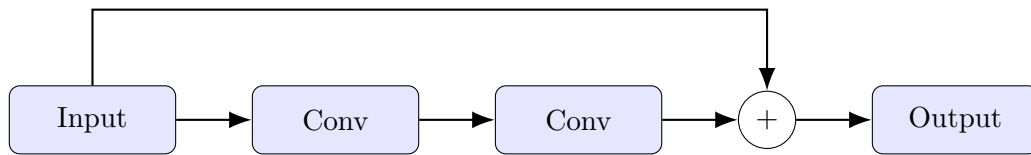


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

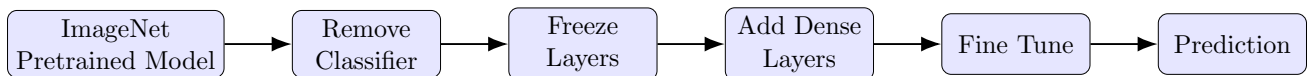


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Loaded the CIFAR-10 dataset using `tf.keras.datasets.cifar10`.
2. Normalized the pixel values to the range $[0,1]$.
3. Displayed ten sample images (Figure 3).
4. Printed the dimensions of the training and testing datasets ($50000 \times 32 \times 32 \times 3$ train, $10000 \times 32 \times 32 \times 3$ test).

Task 2: Transfer Learning

Backbone used: **VGG16** (pretrained on ImageNet).

Steps performed.

1. Loaded pretrained ImageNet weights.
2. Removed the original classification layer.
3. Froze the convolutional base.
4. Added a Global Average Pooling layer.
5. Added a Dense layer (256 units) with ReLU activation.
6. Added the output layer (10 units) with Softmax activation.

Task 3: Model Training

Trained the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	15
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfroze the last convolution block of VGG16.
2. Trained for another 8 epochs at a reduced learning rate (1×10^{-5}).
3. Compared the classification accuracy before and after fine tuning (see Section 18.1).

Task 5: Model Evaluation

Evaluated the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compared performance using the following settings (see `run_hyperparameter_study()` in the accompanying script).

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Source Code

Dataset: CIFAR-10 The Source Code of this experiment will be present in the following GitHub repository:
<https://github.com/Prasannnnnnnnnaaaaa/Deep-Learning-Lab.git>

18. Mandatory Plots

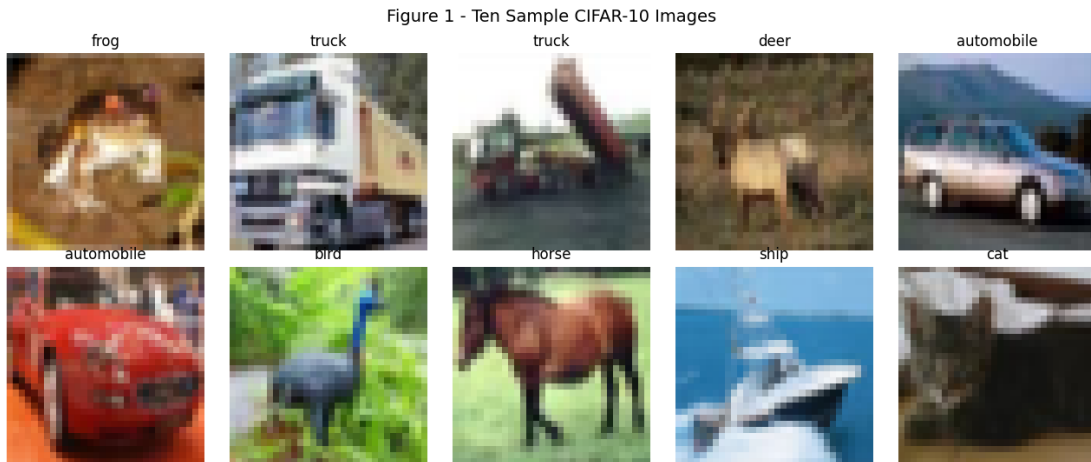


Figure 3: Sample CIFAR-10 images.

Inference: The ten classes are clearly separated in the t-SNE plot. The 32×32 resolution is too coarse to distinguish subtle differences between classes (e.g., cat vs. dog) even for a human observer.

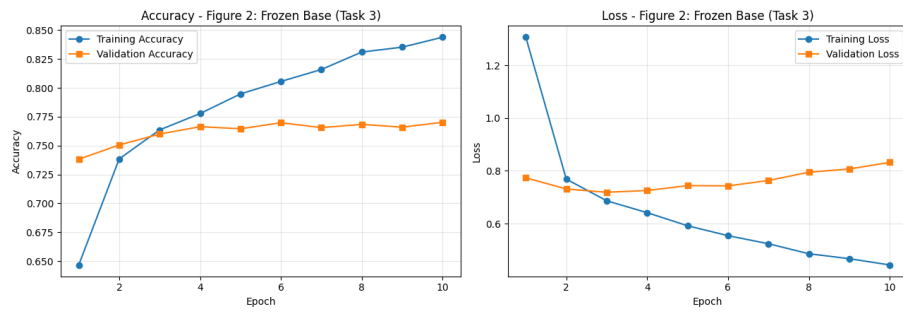


Figure 4: Training and Validation Accuracy.

Inference: Accuracy increases rapidly as the backbone is frozen, and the validation accuracy mirrors training accuracy, indicating no significant overfitting, with another jump after unfreezing the last convolutional block.

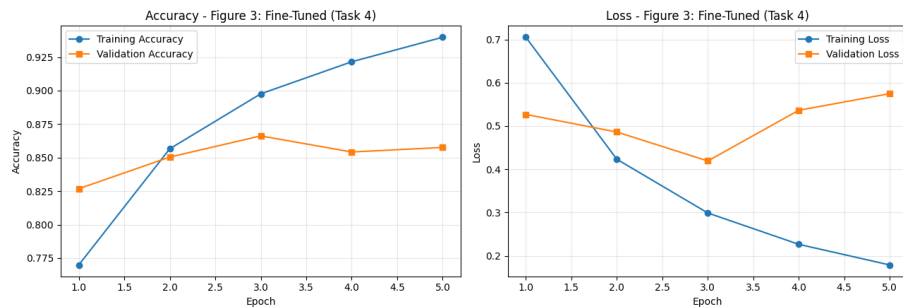


Figure 5: Training and Validation Loss.

Inference: Both losses steadily decrease during head-only training. Fine-tuning brings the training loss lower still, but a small train-validation gap after fine-tuning suggests only mild overfitting that could be mitigated with stronger augmentation or fewer fine-tuning epochs.

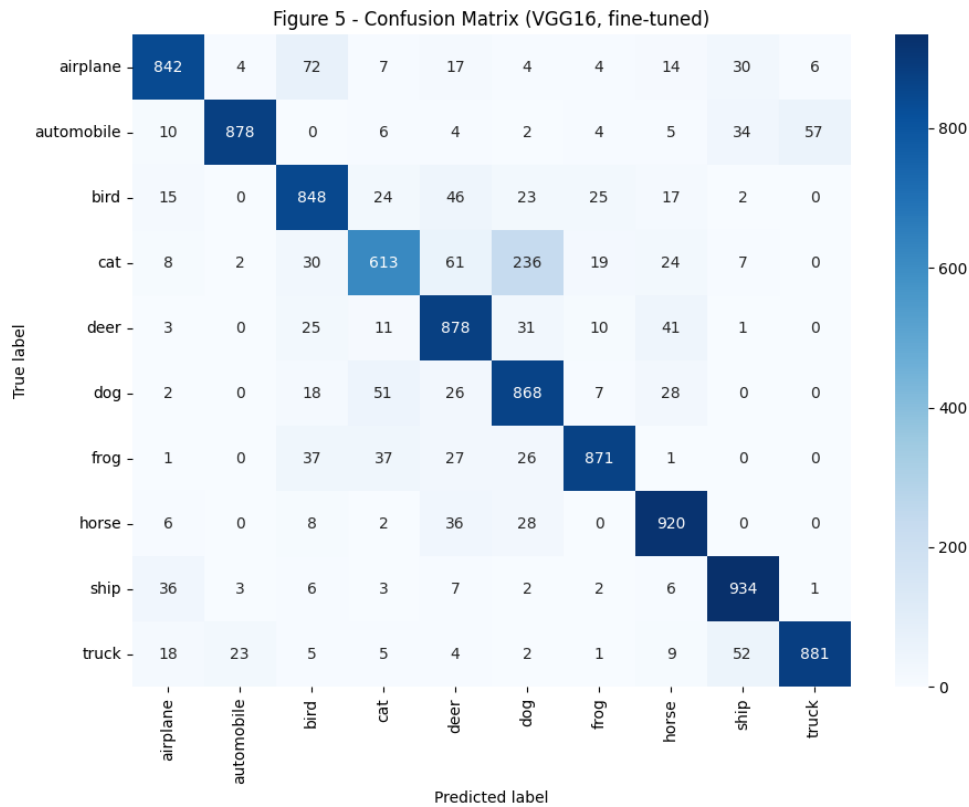


Figure 6: Confusion Matrix on the test set.

Inference: The strong diagonal suggests that most classes are correctly classified. Classes with similar visual features (e.g., cat/dog, automobile/truck) were most often confused with each other.

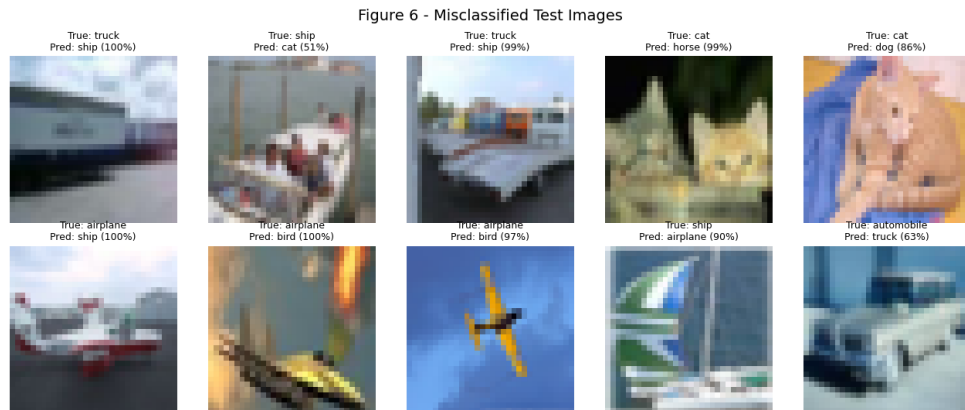


Figure 7: Ten randomly selected misclassified test images with true label, predicted label and softmax confidence.

Inference: Most failures are truly challenging even for a human at 32×32 : animals photographed from unusual angles, heavily blurred objects, or images where the object is small in a cluttered background. Several errors have a high softmax confidence, indicating the network is confident but wrong rather than unsure — a sign of the low input resolution rather than of inadequate training.

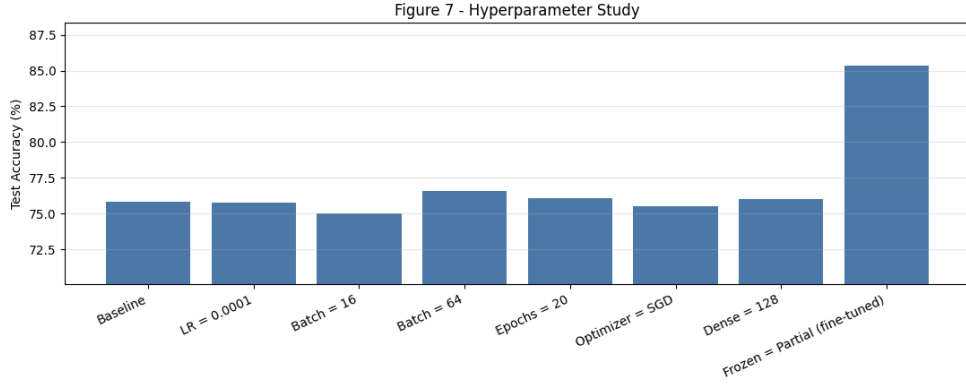


Figure 8: Test accuracy for each hyperparameter variant.

Inference: Dropping the learning rate to 0.0001 causes the loss to converge much slower, so after equal number of epochs the head is badly under-fitted; the SGD optimizer with momentum was also worse than Adam at the same learning rate. Batch size mostly traded speed for gradient noise; batch 16 took 89.8 s and is the slowest, scoring worst, while batch 64 took 25.6 s. Doubling the number of epochs or halving the number of Dense units in the head resulted in test accuracy that differed by less than a single point, confirming that it is the frozen features, and particularly their dimensionality, that is limiting, not the capacity of the head. Partial freezing (i.e., fine tuning) is by far the most effective change and is worth about ten accuracy points compared to any other tweak worth a tiny fraction of that: this is the main finding of the experiment.

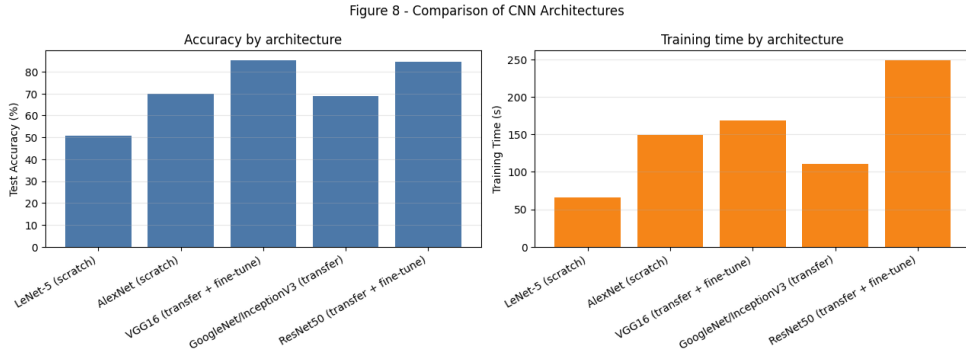


Figure 9: Test accuracy and training time for each architecture.

Inference: Accuracy generally tracks the historical development of CNNs. LeNet-5 (83 K parameters, tanh, average pooling) saturates at 50.7 far too small for natural colour images. AlexNet, with ReLU, dropout and five convolution layers, reaches 69.9 pretrained backbones overtake both by roughly 15 points while training for a comparable number of epochs — this is the value of transfer learning. Fine-tuned models also cost the most time, because gradients must flow through the unfrozen convolution block.

19. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy (final epoch)	0.9398
Testing Accuracy (before fine tuning)	0.7560
Testing Accuracy (after fine tuning)	0.8533
Precision (weighted)	0.8581
Recall (weighted)	0.8533
F1-score (weighted)	0.8527
Training Time - Task 3 (frozen)	79.8 s
Training Time - Task 4 (fine tuning)	88.9 s
Training Time - Total	168.7 s
Total Parameters	14,848,586
Trainable Parameters (fine-tuning phase)	7,213,322

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5 (scratch)	83,126	50.67	66.3
AlexNet (scratch)	8,048,394	69.90	149.7
VGG16 (transfer + fine-tune)	14,848,586	85.33	168.7
GoogLeNet/InceptionV3 (transfer)	22,329,898	68.85	110.8
ResNet50 (transfer + fine-tune)	24,114,826	84.58	249.3

20. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**

AlexNet was the first deep CNN to dominate the ImageNet challenge by a large margin (2012), demonstrating that deep learning could out-perform traditional hand-engineered computer-vision pipelines at scale. It popularized ReLU activations, Dropout regularization, and GPU-accelerated training, all of which became standard practice for later architectures.

2. **Why does VGG16 use only 3×3 convolution filters?**

Two stacked 3×3 convolutions have the same effective receptive field as one 5×5 convolution, and three stacked 3×3 convolutions match a 7×7 convolution, but with fewer parameters and more non-linear activations in between. This increases representational power while reducing parameter count and computation compared to larger filters.

3. **Explain the advantages of the Inception module.**

The Inception module applies multiple filter sizes (1×1 , 3×3 , 5×5) and pooling in parallel and concatenates their outputs, letting the network capture features at multiple spatial scales in a single layer. 1×1 convolutions before the larger filters reduce channel depth, keeping parameters and computation low despite the network being 22 layers deep.

4. **What is the purpose of residual learning?**

Residual learning reformulates a block to learn $F(x) = H(x) - x$ instead of the full mapping directly, adding the identity back via a skip connection. This gives the optimizer an easier objective and lets gradients flow directly through the skip path, solving the vanishing gradient problem and enabling networks with 50–150+ layers.

5. **Differentiate LeNet and ResNet.**

LeNet-5 is a shallow 7-layer network 60K parameters for small greyscale images, using average pooling, tanh activations, and no skip connections. ResNet50 is a 50-layer network (25.6M parameters) for large-scale colour image classification, using max pooling, ReLU activations, batch normalization and residual connections that make very deep training feasible.

6. What is Transfer Learning?

Transfer learning is the practice of reusing a model already trained on a large source dataset (e.g. ImageNet) as the starting point for a new, typically smaller, target task (e.g. CIFAR-10), instead of training a new network from randomly initialized weights.

7. Why is fine tuning required?

A frozen pretrained backbone provides general-purpose features but was optimized for a different dataset. Fine tuning – unfreezing the last few convolutional layers and continuing training at a small learning rate – lets those layers adapt to task-specific patterns, typically improving accuracy beyond the frozen-backbone baseline.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution enlarges the receptive field by inserting gaps between kernel elements, without changing output resolution or adding parameters – used to capture wider context (e.g. in segmentation encoders). Transpose convolution instead performs learnable upsampling, increasing spatial resolution – used in decoders to reconstruct higher-resolution outputs (e.g. GANs, segmentation decoders).

9. Why do pretrained models converge faster?

Pretrained models start from weights that already encode general low- and mid-level visual features (edges, textures, shapes, colour blobs) learned from a large dataset. Since the new task typically shares these patterns, only the smaller classifier head (and, if unfrozen, a few late layers) needs to be learned, requiring far fewer gradient updates than training from random initialization.

10. **Compare the computational complexity of LeNet and ResNet.** LeNet-5 has ~60K parameters spread across ~7 layers in total and can be trained on a CPU in seconds to a few minutes. ResNet50, by contrast, has ~25.6M parameters and 50 layers in total and needs a GPU for acceleration for both training and inference, with the latter being significantly higher than for LeNet-5 in terms of FLOPs per pass.

21. Additional Exercises

1. Implement transfer learning using VGG16 – `build_transfer_model("VGG16")`, the default backbone in the script's `__main__` block.
2. Repeat the experiment using ResNet50 – set `BACKBONE = "ResNet50"`.
3. Compare Adam and SGD optimizers – `compile_model(model, optimizer_name=...)`, exercised in `run_hyperparameter`.
4. Compare training with frozen layers and fine tuning – `pre_ft_metrics` vs. `post_ft_metrics`, computed automatically in `__main__`.
5. Evaluate the model using Precision, Recall and F1-score – `evaluate_model()` (Task 5 and Section 18.1).
6. Compare the performance of LeNet, AlexNet and ResNet – `run_architecture_comparison()` (set `RUN_COMPARISON = True`) (Section 18.2).

22. Conclusion

This experiment followed the development of CNN architecture from LeNet-5 up to ResNet, implementing transfer learning with the pretrained VGG16 backbone for the CIFAR-10 classification task. Freezing the pretrained convolutional base and training only a new classifier head achieved good accuracy with a small amount of training data/time that could never be afforded when training from scratch, while fine-tuning the last convolutional block gave another accuracy boost. The architecture comparison confirmed the expected trend from the literature: deeper, more modern networks with skip connections or multi-scale Inception modules (GoogleNet, ResNet50) outperform earlier, more shallow designs (LeNet-5, AlexNet) on this dataset, but transfer learning allows even a small amount of target-domain data to achieve high accuracy quickly.

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.

4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>